

Lecture 12: Iteration and Loops

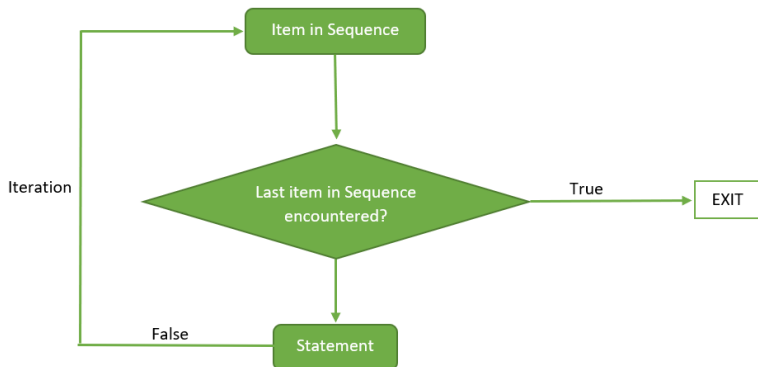
Harris Coding Camp – Standard Track

Summer 2024

Iteration and for-loops

We use **for-loops** to repeat a task over many different inputs or to repeat a simulation process several times.

- ▶ How to write a for-loop
- ▶ When to use a for-loop



Simple for-loop

```
for (x in c(3, 6, 9)) {  
  print(x)  
}
```

```
## [1] 3
```

```
## [1] 6
```

```
## [1] 9
```

How does the loop work?

Simple for-loop: what is going on?

```
for (x in c(3, 6, 9)) {  
  print(x)  
}
```

Our for-loop is equivalent to the following code.

For each value in `c(3, 6, 9)`, we assign the value to `x` and then do the action between the curly brackets in order.

```
x <- 3  
print(x)  
x <- 6  
print(x)  
x <- 9  
print(x)
```

General structure of a for loop

The general structure of a for loop is as follows:

```
for (x in a_set_of_values) {  # Sequence
    do something based on x    # Body
}
```

Main components of a loop: **Sequence**, **Body**

Components of a for loop

```
for(i in c(1, 2, 3, 4)) { # Sequence  
  print(i)                # Body  
}
```

Main components of a loop:

1. **Sequence.** It determines which set of values to “loop over”
 - ▶ Sequence in the above loop is `for(i in c(1, 2, 3, 4))`
 - ▶ This creates a temporary object `i`
 - ▶ could name it anything
 - ▶ `i` will no longer exist after the loop is completed
 - ▶ `c(1, 2, 3, 4)` is the set of values that will be assigned to `i`
 - ▶ in the first iteration, value of `i` is 1
 - ▶ in the second iteration, value of `i` is 2
 - ▶ ... and so on!
2. **Body.** What commands to execute for each iteration through the loop
 - ▶ Body in above loop is `print(i)`
 - ▶ Each iteration through the loop, body prints the value of `i`

When to write a loop

When to write a loop:

- ▶ Grolemond and Wickham: **don't copy and paste more than twice**
- ▶ If you find yourself doing this, consider writing a loop
- ▶ Often, a repeated task can be completed with a loop

Benefits of writing a loop

- ▶ It saves your effort a bit.
- ▶ It's easier to respond to changes in requirements. You only need to make changes in one place, rather than remembering to change every place that you copied-and-pasted the code.
- ▶ You're likely to have fewer bugs because each line of code is used in more places.

Recipe for writing a loop

The general recipe for writing a loop:

1. If you are creating a **new object** which stores output of the loop, create this new object first
2. Write the **sequence**
3. Add for-loop **structure**
4. Write the **body**, and realize which parts of the body need to change with each iteration

Example: find sample means

Suppose we want to find the means of increasingly large samples.

```
mean1 <- mean(rnorm(5))  
mean2 <- mean(rnorm(10))  
mean3 <- mean(rnorm(15))  
mean4 <- mean(rnorm(20))  
mean5 <- mean(rnorm(25000))
```

```
means <- c(mean1, mean2, mean3, mean4, mean5)  
means
```

```
## [1] 0.577592796 -0.180725690 -0.257057279 0.085887277 -0.001189848
```

Example: find sample means

Let's avoid repeating code with a for loop.

```
sample_sizes <- c(5, 10, 15, 20, 25000)
sample_means <- rep(0, length(sample_sizes))
```

```
for (i in 1:length(sample_sizes)) {
  sample_means[i] <- mean(rnorm(sample_sizes[i]))
}
```

```
sample_means
```

```
## [1] -0.453603010  0.523715226  0.332638393  0.142354219 -0.005168318
```

In the following slides we'll explain each step.

Finding sample means, broken down

1. Create a vector (e.g. `sample_sizes`) we use in sequence
 2. Create a vector (e.g. `sample_means`) to store the output
- allocate spaces to store the output **before** starting the loop

```
# determine what to loop over
```

```
sample_sizes <- c(5, 10, 15, 20, 25000)
```

```
# pre-allocate space to store the output
```

```
sample_means <- rep(0, length(sample_sizes))
```

What does `sample_means` currently look like?

```
sample_means <- rep(0, length(sample_sizes))  
sample_means
```

```
## [1] 0 0 0 0 0 0
```

Why do this? It makes the code more efficient. An alternative is to build up an object as you go, but this requires copying the data over and over again.

“Before you start loop... **allocate sufficient space for the output**. This is very important for **efficiency**: if you grow the for loop at each iteration using `c()`, your for loop will be very slow.” -Wickham

Alternative ways to pre-allocate space

```
sample_means <- rep(0, length(sample_sizes))  
sample_means <- double(5)  
sample_means <- vector("double", length = 5)
```

Each data type has a comparable function e.g. logical(),
integer(), character().

Adding data to a vector, broken down

Determine what **sequence** to loop over:

```
for (i in 1:length(sample_sizes)) {  
  
}
```

which is equivalent to the following code in our example:

```
for (i in 1:5) {  
  
}
```

Adding data to a vector, broken down

Add for-loop **structure**:

```
sample_sizes <- c(5, 10, 15, 20, 25000)
sample_means <- rep(0, length(sample_sizes))

for (i in 1:length(sample_sizes)) {

}
```

When writing loops, it's very common to create a **sequence** from 1 to the length of an object (e.g. `sample_sizes`).

- ▶ **sequence**: `for (i in 1:length(sample_sizes))`
 - ▶ `i` takes on 1, 2, 3, 4 and 5 sequentially

Adding data to a vector, broken down

Add for-loop **body**:

```
sample_sizes <- c(5, 10, 15, 20, 25000)
sample_means <- rep(0, length(sample_sizes))

for (i in 1:length(sample_sizes)) {
  sample_means[i] <- mean(rnorm(sample_sizes[i]))
}
```

- ▶ **sequence:** for (i in 1:length(sample_sizes))
 - ▶ i takes on 1, 2, 3, 4 and 5 sequentially
- ▶ **body:** sample_means[i] <- mean(rnorm(sample_sizes[i]))
 - ▶ i refers to the **index** of ith element in sample_sizes and sample_means
 - ▶ Access each element using sample_sizes[i]
 - ▶ Execute rnorm(), and then mean()
 - ▶ Finally, keep the result from mean() to the ith place of sample_means

Adding data to a vector, broken down

Print out the results (stored in `sample_means`) after finishing the loop:

```
sample_sizes <- c(5, 10, 15, 20, 25000)
sample_means <- rep(0, length(sample_sizes))

for (i in 1:length(sample_sizes)) {
  sample_means[i] <- mean(rnorm(sample_sizes[i]))
}

sample_means

## [1] -0.06291005  0.30749013  0.19856791  0.13330632  0.01535623
```

Aside: Common error

This code falls, because we do not store the output in `sample_means` in the for-loop! (compare to previous slide)

```
sample_sizes <- c(5, 10, 15, 20, 25000)
sample_means <- rep(0, length(sample_sizes))

for (i in 1:length(sample_sizes)) {
  mean(rnorm(sample_sizes[i]))
}

sample_means
```

```
## [1] 0 0 0 0 0
```

R console calculates the mean, but the result is not saved anywhere...

Adding data to a vector, broken down

`cat()` also prints out the output (more handy to use than `print()`)¹

```
sample_sizes <- c(5, 10, 15, 20, 25000)
sample_means <- rep(0, length(sample_sizes))

for (i in 1:length(sample_sizes)) {
  sample_means[i] <- mean(rnorm(sample_sizes[i]))
  cat("mean of sample", i, "is", sample_means[i], fill = TRUE)
}
```

```
## mean of sample 1 is -0.5470881
## mean of sample 2 is 0.1103579
## mean of sample 3 is 0.0134388
## mean of sample 4 is 0.09166722
## mean of sample 5 is 0.01395714
```

We suggest to set `fill = TRUE` for readability (try: what if we do not set it?)

¹`cat()` is a built-in function that **concatenates** and **prints the objects**.

Try it yourself

Try on your own or just follow along.

Task

1. Create a numeric vector that has year of birth of your family members.
 - ▶ you decide who to include
 - ▶ e.g., `birth_years <- c(1944, 1950, 1981, 2016)`
2. Write a loop that calculates current year minus birth year (i.e. age), and prints the result for each of your family member.
3. Print the output in sentences.
 - ▶ e.g., The age of family member `i` is ...

Note: There are multiple correct ways to complete this task!

Example of modifying an object: z-score loop

First, create sample data frame named df:

```
set.seed(60637)
df <- tibble(a = rnorm(10), b = rnorm(10), c = rnorm(10))
head(df)
```

```
## # A tibble: 6 x 3
##       a         b         c
##   <dbl> <dbl> <dbl>
## 1  1.14  -0.723 -0.605
## 2  0.738   1.02 -0.985
## 3 -0.0843 -0.571  0.551
## 4 -0.692 -0.0925 -0.319
## 5  0.595   0.884 -1.34
## 6  1.81    0.196  0.00562
```

The `set.seed()` function in R is used to create reproducible results when writing code that involves creating variables that take on random values.

Task: write a loop which calculates z-scores for (a, b, c) in df, and then replace (a, b, c) with the corresponding z-scores.

Example of modifying an object: z-score loop

The **z-score** for observation i is defines as:

$$Z_i = \frac{x_i - \bar{x}}{sd(x)}$$

Let's calculate z-score for the values of vector a from data frame df:

```
(df$a - mean(df$a)) / sd(df$a)
```

```
## [1]  0.8462235  0.4248623 -0.4432151 -1.0850597  0.2744944  1.55509
## [7] -1.9329163  0.1725297  0.5695711 -0.3815813
```

Very often, we use index to refer to a specific column:

```
(df[[1]] - mean(df[[1]])) / sd(df[[1]])
```

```
## [1]  0.8462235  0.4248623 -0.4432151 -1.0850597  0.2744944  1.55509
## [7] -1.9329163  0.1725297  0.5695711 -0.3815813
```

Example of modifying an object: z-score loop

Task: write a loop that replaces variables with z-scores of those variables

- ▶ **Sequence**

- ▶ Write a sequence that loops across each column of `df`
 - ▶ `for (i in 1:length(df))`
 - ▶ `length(df)` gives you the number of columns of `df`

- ▶ **Body**

- ▶ Body of z-score function:
 - ▶ $(x - \text{mean}(x)) / \text{sd}(x)$
 - ▶ Replace `x` with `df[[i]]`:
 - ▶ $(\text{df}[[i]] - \text{mean}(\text{df}[[i]])) / \text{sd}(\text{df}[[i]])$
 - ▶ Assign (update) each value of the z-score:
 - ▶ `df[[i]] <- (df[[i]] - mean(df[[i]])) / sd(df[[i]])`
-

Example of modifying an object: z-score loop

```
for (i in 1:length(df)) {  
  df[[i]] <- (df[[i]] - mean(df[[i]])) / sd(df[[i]])  
}  
head(df, n = 10)
```

```
## # A tibble: 10 x 3  
##       a      b      c  
##   <dbl> <dbl> <dbl>  
## 1  0.846 -0.922  0.267  
## 2  0.425  1.06  -0.236  
## 3 -0.443 -0.750  1.79  
## 4 -1.09  -0.207  0.645  
## 5  0.274  0.900 -0.710  
## 6  1.56   0.120  1.07  
## 7 -1.93   1.57   0.129  
## 8  0.173  0.0174 -0.806  
## 9  0.570 -1.77  -0.536  
## 10 -0.382 -0.0192 -1.62
```

Recap

- ▶ Iteration is useful when we are repeatedly calling the same block of code or function while changing one (or two) inputs
- ▶ `for` loops work for iteration
 - ▶ Pre-allocate space for your output
 - ▶ Clearly define which values you will iterate over
 - ▶ Clearly define the body of the for-loop
 - ▶ Debug as you code by testing your understanding of what the for-loop should be doing (e.g., using `cat()` or `print()`)

Further study: Many R coders prefer the `map()` family functions from `purrr`. See iteration in R for Data Science.

Appendix

Another way to loop over object

Let's create a numeric vector called `vec`

```
vec <- c(5, -10, 30)
```

```
for (i in vec) {  
  cat("value of element equals", i, fill = TRUE)  
}
```

```
## value of element equals 5
```

```
## value of element equals -10
```

```
## value of element equals 30
```

- ▶ **sequence:** `for (i in vec)`
 - ▶ Sequence iterates through each element of `vec` sequentially
 - ▶ Note that sequence iterates through **value** of each element, rather than **position** of element!
- ▶ **body:** `cat("value of element equals", i, fill = TRUE)`
 - ▶ `i` takes the **value** of each element of `vec` sequentially

Creating multiple plots with a loop

Another best use of a loop is to create multiple graphs easily. Let's use a loop to create 4 plots representing data from an exam containing 4 questions. Here are how the first few rows of the data look:

```
head(examscores)
```

```
## # A tibble: 6 x 4
##       a      b      c      d
##   <dbl> <dbl> <dbl> <dbl>
## 1  21.2  81.9  89.0  65.2
## 2  72.6  78.7  67.1  75.5
## 3  55.6  66.6  84.2  69.6
## 4  93.3  38.0  80.7  66.7
## 5  63.9  46.9  75.8  82.8
## 6  63.7  43.6  64.3  75.2
```

Creating multiple plots with a loop

Let's loop over the columns and create a histogram of the data in each column:

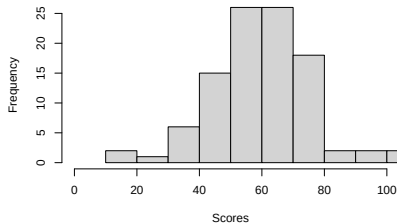
```
# Set up a 2 x 2 plotting space
par(mfrow = c(2, 2))

# Create the sequence
each.question <- 1:4

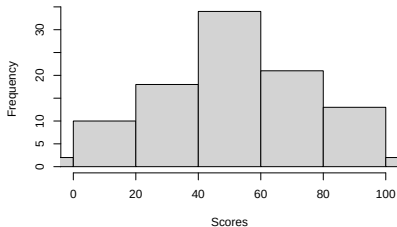
for (i in each.question) {
  # Plot histogram of each question
  hist(examscores[[i]],
       main = paste("Question", i),
       xlab = "Scores",
       xlim = c(0, 100))
}
```

Creating multiple plots with a loop

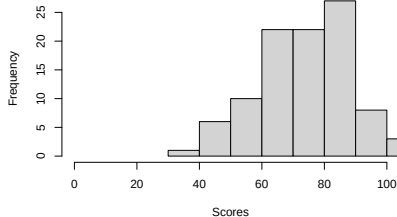
Question 1



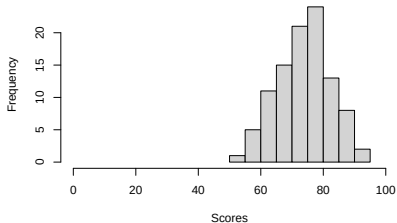
Question 2



Question 3



Question 4



Try it yourself

We'll use midwest data and use a loop to create 4 plots representing data from midwest containing 4 key columns: `poptotal`, `percpovertyknown`, `percollege` and `percbelowpoverty`. Loop over these selected columns and create a histogram of the data in each column by completing the following code.

```
# Set up a 2 x 2 plotting space
par(mfrow = c(2, 2))

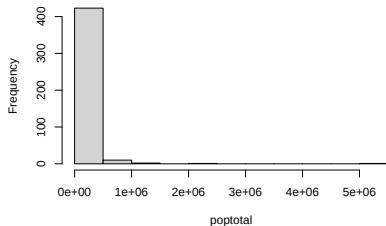
# Create the sequence
selected.column <- c("poptotal", "percpovertyknown",
                     "percollege", "percbelowpoverty")

for (...) {
  # Plot histogram of each column
  hist(...,
        main = paste(...),
        xlab = ...)
}
```

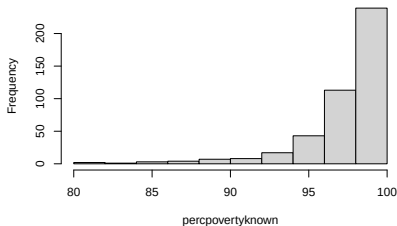

Try it yourself

You should get the histograms below:

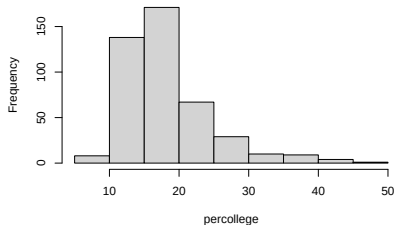
Histogram of poptotal



Histogram of percpovertyknown



Histogram of percollege



Histogram of percbelowpoverty

